

1.3 — Configure Subagent Invocation, Context Passing, and Spawning

Concept

In multi-agent systems, an orchestrator agent spawns subagents to handle discrete subtasks. The **Task tool** in the Claude Agent SDK is the primary mechanism for this: the orchestrator calls `Task` with a description of the work, an optional set of allowed tools, and any context the subagent needs to execute independently. The SDK handles launching a fresh agent loop for that subagent, running it to completion, and returning the result to the orchestrator.

Tool scoping via `allowedTools` is critical for security and predictability. Each subagent should only have access to the tools it actually needs. A subagent responsible for reading files should not have write or shell execution tools. This principle of least privilege prevents runaway agents from taking destructive actions and makes the system easier to reason about. If `allowedTools` is omitted, the subagent inherits all tools available in the environment — a common source of unintended side effects.

Context isolation is the other essential discipline. Subagents do not share memory or conversation state with the orchestrator or with each other. Any information a subagent needs — user intent, prior results, relevant file contents, configuration — must be passed explicitly in the task prompt. Relying on shared global state or assuming a subagent "remembers" something from the parent conversation is a design error that produces silent, hard-to-debug failures. When you need to run multiple subagents over the same data, pass that data explicitly to each one. Parallel execution with `Promise.all` is both safe and efficient when subagents are truly independent, since each runs in its own isolated context.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// Orchestrator spawns two parallel subagents with scoped tools and explicit context
async function analyzeRepository(repoSummary: string): Promise<void> {
  const [securityReport, qualityReport] = await Promise.all([
    // Subagent 1: security review — read-only file access only
    client.beta.messages.create({
      model: "claude-sonnet-4-6",
      max_tokens: 4096,
      tools: [
```

```

    {
      type: "computer_20241022",
      name: "Task",
      description: "Spawn a subagent for security review",
    },
  ],
  messages: [
    {
      role: "user",
      content: `You are a security reviewer. Analyze this repository summary for vulnerabi

```

Repository context:

`${repoSummary}`

Use only `read_file` to inspect individual files. Do not write or execute anything.

Return a structured JSON report with findings.`,

```

    },
  ],
  betas: ["computer-use-2024-10-22"],
}),

```

// Subagent 2: code quality – read-only, no security concerns

```

client.beta.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 4096,
  messages: [
    {
      role: "user",
      content: `You are a code quality reviewer. Analyze this repository summary for mainta

```

Repository context:

`${repoSummary}`

Focus on: naming conventions, file size, cyclomatic complexity, dead code.

Return a structured JSON report with findings.`,

```

    },
  ],
}),
);

```

// Aggregate results from both isolated subagents

```
console.log("Security findings:", securityReport.content);
```

```
console.log("Quality findings:", qualityReport.content);
```

```
}
```

// Explicit context passing – never rely on shared state

```
async function processChunks(chunks: string[]): Promise<string[]> {
```

```

return Promise.all(
  chunks.map((chunk, index) =>
    client.messages
      .create({
        model: "claude-haiku-4-5",
        max_tokens: 1024,
        messages: [
          {
            role: "user",
            // Each subagent receives its full context explicitly
            content: `Process chunk ${index + 1} of ${chunks.length}:

${chunk}

Return a one-paragraph summary.`
          },
        ],
      })
      .then((r) => r.content[0].type === "text" ? r.content[0].text : ""))
  );
}

```

Common Mistakes

1. **Omitting `allowedTools` on subagents** — Without explicit scoping, subagents inherit all available tools. A summarization subagent that accidentally has shell execution access can cause unintended side effects. Always declare the minimal set of tools each subagent requires.
2. **Passing context by reference or shared state** — Subagents run in isolated loops with no access to the orchestrator's conversation history. Storing shared context in a global variable and expecting subagents to read it silently fails. Every piece of information a subagent needs must appear in its initial `messages` array.
3. **Running independent subagents sequentially** — Using `await` in a loop when subagents do not depend on each other's results wastes wall-clock time proportional to the number of agents. Use `Promise.all` (or `Promise.allSettled` when partial failures are acceptable) to run truly independent subagents concurrently.

References

- [Anthropic: Build multi-agent systems](#)
- [Anthropic: Tool use overview](#)
- [Anthropic: Model context and memory](#)

